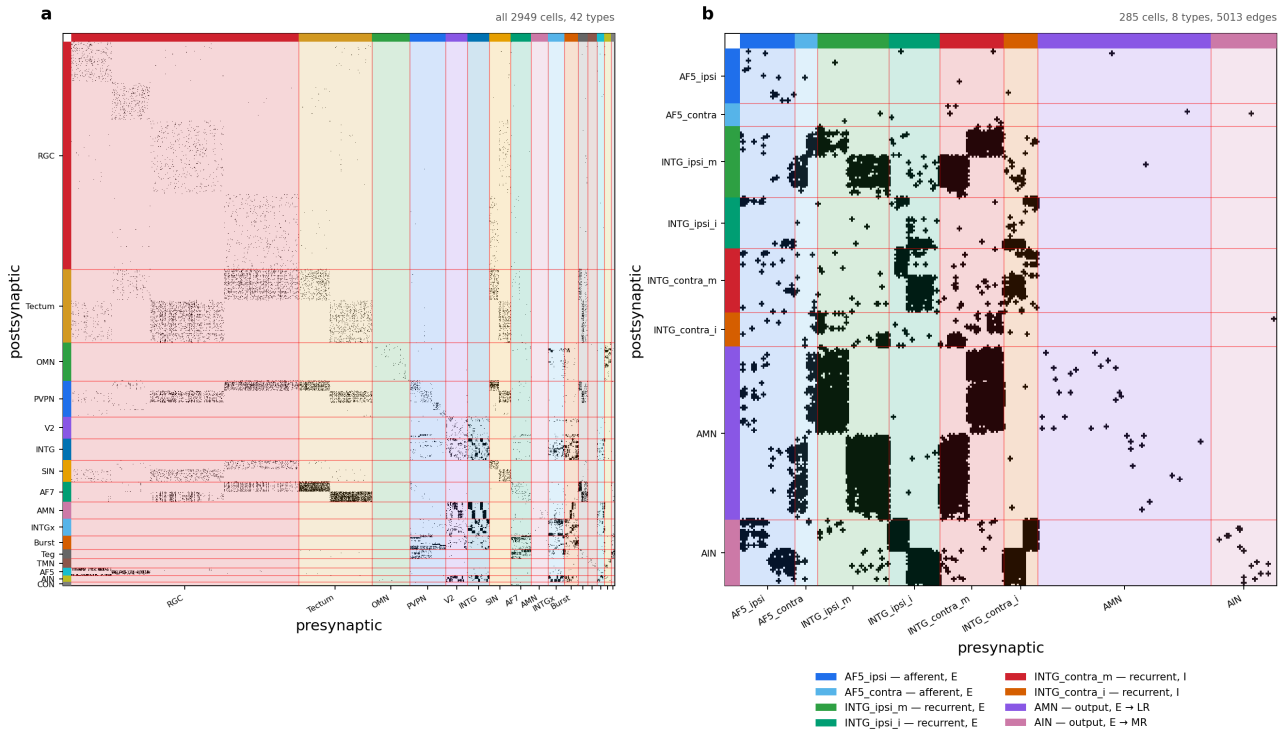
The larval zebrafish oculomotor integrator: circuit, task, and training data

2026-08-12

The larval zebrafish oculomotor integrator: circuit, task, and training data

Working document for the `feat/oculomotor` branch. It records what the circuit is taken to be, what the network will be asked to compute, and how the training data is generated. Everything here that is a *claim about biology* rather than a fact read off the reconstruction is marked as such — those are the parts that need the circuit owner's signature before any result means anything.

Source reconstruction: `Oculomotor_sortedData_081126.pkl`, 2949 cells, 42 cell types, 44,584 edges (density 0.51 %). Selected sub-circuit: `config/zebrafish/zebrafish_om_intg_285_v1.yaml`, 285 cells, 8 types.



The oculomotor reconstruction and the sub-circuit modelled here. (a) All 2949 cells, ordered by the 16 coarse cell-type families, support mask of the synapse-area matrix. (b) The 285-cell sub-circuit, ordered afferent then recurrent then output, and within a type left hemisphere before right. Colour carries the biology: blue = afferent, green = excitatory recurrent, red/orange = inhibitory recurrent, purple/pink = motor output. Rows are postsynaptic, columns presynaptic. Regenerate with `python scripts/plot_oculomotor_connectome.py -config config/zebrafish/zebrafish_om_intg_285_v1.yaml -out figures/zebrafish/fig_oculomotor_circuit.png`.

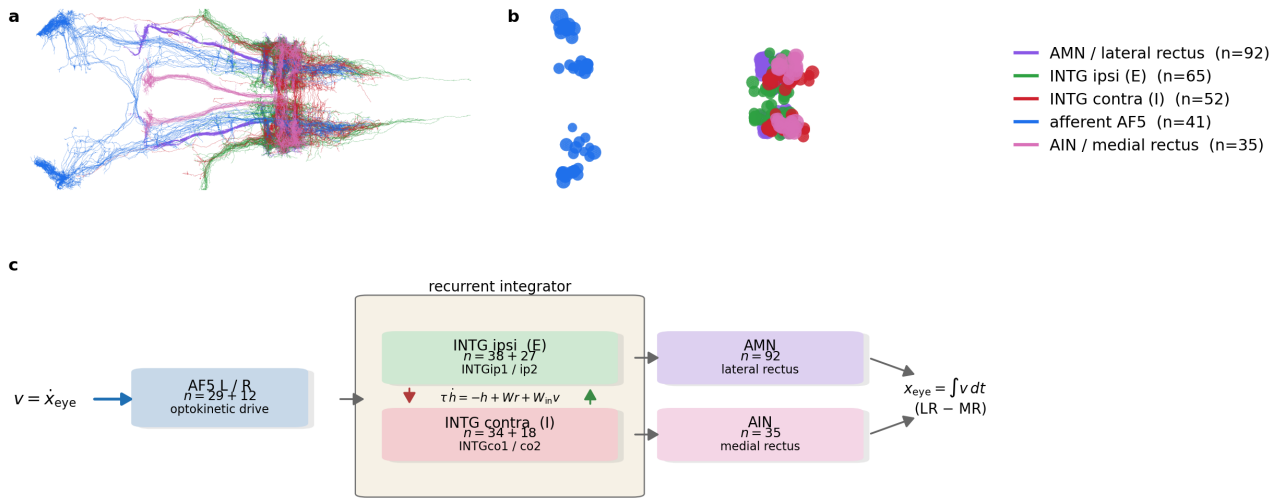


Figure 1 — the oculomotor circuit. Twin of Figure 1 of the zebrafish heading-direction paper, rendered through the same code with the content swapped. **(a)** All 285 skeletons from the neuprint-fish2 reconstruction, dorsal view, coloured by role: AF5 afferents blue, ipsilateral (excitatory) INTG green, contralateral (inhibitory) INTG red, AMN purple, AIN pink. **(b)** The cell bodies alone, same view — the AF5 somas sit well anterior of the integrator/motor cluster (radii scaled x0.3 for legibility, not a measurement). **(c)** The computation: an optokinetic velocity signal enters through AF5, the INTG pool integrates it under a continuous-time rate dynamics with mutual inhibition across the midline, and eye position is read out as the push-pull difference of the two motor pools. Regenerate with `python figures/zebrafish/fig_1_oculomotor_overview.py`.

1. The circuit

1.1 Three pools

Anatomy for all 285 cells was fetched from neuprint-fish2 and is shown in Figure 1. It had to be keyed on bodyId rather than cell type: the server does not carry the reconstruction's type names — `INTG_ipsi_m` is `INTGip1` there, `INTG_contra_m` is `INTGco1` — and the two AF5 populations carry no type at all, so a type query returns 142 of 285 cells and drops the whole afferent stage. All 285 bodyIds resolve.

The 285 cells divide into an input stage, a recurrent integrator and a motor output stage.

pool	types	cells	Dale sign
afferent	`AF5_ipsi` (29), `AF5_contra` (12)	41	excitatory
recurrent	`INTG_ipsi_m` (38), `INTG_ipsi_i` (27), `INTG_contra_m` (34), `INTG_contra_i` (18)	117	ipsi excitatory, contra inhibitory
output	`AMN` (92), `AIN` (35)	127	excitatory

Within the sub-circuit there are 5013 edges, a density of 6.2 % — an order of magnitude above the 0.51 % of the full reconstruction. That concentration is what one expects of a recurrent integrator core plus its dedicated input and output stages, and it is the first (weak) evidence that the selection is a circuit rather than an arbitrary subset.

1.2 AF5 is an afferent, and the wiring says so

The two AF5 populations are the pretectal arborization field that carries optokinetic drive. Their afferent status is not assumed. In the measured matrix, AF5 sends 88.2 % of its outgoing synaptic area into the INTG/AMN/AIN core and receives 1/178th as much back (6.89e5 out against 3.87e3 in). This is the same

asymmetry test that confirmed the matrix orientation using the retinal ganglion cells, which are 15.4x more outgoing than incoming.

Both AF5 types are left/right balanced — `AF5_ipsi` 15 L / 14 R, `AF5_contra` 6 L / 6 R — so a bilateral input gate is expressible. This is worth stating because it is not generally true in this dataset: `RGC_AF7` is reconstructed in the left hemisphere only (804 L / 0 R) and could not support a symmetric gate.

1.3 What drives the afferents — CLAIM, and an open one

The direction selectivity below is the circuit owner's description, not a measurement in this file. The combination rule is explicitly unresolved.

- `AF5_ipsi` — the left pool is driven by rightward-eye motion that is

leftward and/or forward; the right pool by leftward-eye motion that is rightward and/or forward.

- `AF5_contra` — the left pool is driven by rightward-eye motion that is

rightward and/or backward; the right pool by leftward-eye motion that is leftward and/or backward.

Whether the two conditions combine as AND, OR or XOR is **not known**. This is not a detail to be settled by a default: it decides whether a single afferent unit reports a conjunction (a narrow direction tuning) or a disjunction (a broad one), and therefore how much of the direction computation the recurrent circuit has to do itself. Until it is resolved, the input model should carry it as an explicit switch and the three variants should be trained and compared.

1.4 The integrator, and why the E/I split is by laterality

The four INTG types are the velocity-to-position integrator. The sign assignment follows their projection laterality: **ipsilateral projections excitatory, contralateral projections inhibitory**. That is the classical integrator motif — two half-integrators, one per hemisphere, each self-exciting locally and inhibiting its opposite number across the midline. Mutual inhibition is what lets the pair hold a *signed* position variable on a single positive-rate substrate.

This is a Dale assignment by cell type, exactly as in the heading-direction circuit, where `_ZHD_INH_PREFIXES` in `connectome_loaders.py` encodes the claim that the r1pi/dIPN cells are GABAergic. Neither is a measurement. The difference is that here the claim is written in the config, per type, where it can be reviewed and flipped, rather than in a tuple of string prefixes inside a loader.

1.5 The output stage, and one simplification worth flagging

Only the **horizontal** pair of extraocular muscles is modelled:

- `AMN` drives the **lateral rectus** (`LR` in the plant), which abducts the

eye;

- `AIN` drives the **medial rectus** (`MR`), which adducts it.

Muscle keys refer to the six-muscle soft-body eye in `Plexus/prototype/eye`, where each muscle carries an innervation state `muscle.act` and the globe's pose is recovered by a Kabsch fit (`eye_pose` -> horizontal, vertical, torsion in degrees). Coupling the readout to that plant, rather than to an abstract scalar, is what would make "eye position" a mechanical quantity instead of a fitted one.

The simplification. `AIN` are abducens *internuclear* neurons. In vivo they are not motor neurons: they project across the midline to the oculomotor nucleus (`OMN`), whose motor neurons innervate the medial rectus. `OMN` is 207 cells in this reconstruction and is **not** in the selected pool. Treating `AIN` as driving the medial rectus therefore collapses a two-synapse pathway into one, and drops the sign inversion and delay that the missing synapse would contribute. This is a legitimate modelling choice, but it should be stated in any write-up, and the obvious control is to add `OMN` to `circuit.cell_types` and check whether the fitted dynamics change.

1.6 What is not in the pool

Two omissions are deliberate and one of them constrains the task:

- `Burst_T1A/T1B/T8/T9/T10` (74 cells) are the saccadic burst

generators — the source of the brief velocity command that a saccadic integrator integrates. They are excluded, so in the current pool the only anatomically defined input is the optokinetic AF5 drive. A saccade-driven task would have to inject velocity directly into INTG, which is a modelling choice, not a connectome-derived pathway.

- `OMN` (207 cells), as described in §1.5.

2. The task

2.1 What the network computes

The oculomotor integrator converts an eye-**velocity** command into an eye- **position** command: the motor neurons must hold a rate proportional to position long after the velocity signal has gone. Written as the leaky integral of the drive,

dP/dt = -P/tau + v(t)

the quantity of interest is `tau`, the time constant over which position leaks back to centre. A perfect integrator is `tau = infinity`; the biological integrator is finite, and measuring what `tau` the connectome-constrained circuit can support is the point of the exercise.

2.2 It needs no new task code

This is already a mode of the existing swim-integration generator. Setting

task.swim_integration.target_kind: scalar_xi task.swim_integration.xi_tau_s: <the leak> training.task_-targets: [translation]

yields a **1-input / 1-output** problem: the input channel is the velocity drive, the target is `xi = integral of v dt` with leak `xi_tau_s`. The `_integrate_leaky` helper in `graph_data_generator.py` implements exactly the equation above (forward Euler, `alpha = 1 - dt/tau`), and `tau <= 0` recovers the perfect integrator.

What the existing machinery does **not** provide is the neuron binding: which cells the input enters and which cells the readout reads. In the heading circuit both are hardwired to the HD taxonomy — the encoder through `graph_model.velocity_gate`, the decoder through `output_from_dipn_only` and a positional `[:n_bump]` slice. Redirecting them to the AF5 afferents and the AMN/AIN outputs is the real work, and it is Procedure B in `HOWTO_add_zebrafish_circuit.md`.

2.3 Saccades versus optokinetic drive

The two natural stimulus regimes differ, and the choice interacts with §1.6:

- **Optokinetic**: sustained whole-field motion, the drive AF5 actually

carries. Slow, continuous velocity — the regime the selected pool supports anatomically.

- **Saccadic**: brief high-velocity pulses separated by fixations, the

classical integrator probe. This is the regime the burst generators serve, and they are not in the pool.

The swim generator's stimulus is a Poisson train of boxcar impulses (`swim_rate_hz`, `swim_duration_s`) — structurally a saccade train already. So the saccadic regime is reachable by reparameterising the existing generator rather than writing a new one; what is missing is anatomy for the input, not code.

2.4 The open-loop problem

Coarsely: the circuit is asked to keep a moving target centred on the fovea while being told only how fast the target is moving, never where it is. That is the situation the anatomy puts it in. AF5 is a motion-sensitive arborization field — it reports optokinetic *velocity* — and nothing in the selected 285-cell pool reports absolute

target position. A controller with position feedback can always correct, so its errors do not accumulate; a controller given velocity alone must reconstruct position by integration and has nothing to check the result against. Drift is therefore not a failure of the circuit but a property of the problem, and the meaningful question is not whether the target is held but for how long.

More specifically, write the target position `p(t)`, the gaze `g(t)`, and the retinal error `e = p - g`. Every trial starts foveated at the centre, `g(0) = p(0) = 0`. The plant is a rate control: the motor command sets gaze *velocity*, so gaze position is its integral. The ideal controller is then

dg/dt = v(t) , v = dp/dt -> e(t) = 0 for all t

and tracking is exact. The interesting cases are the three ways a biological integrator departs from it, each of which produces a different growth law for `|e|` — which is what makes them separable from a single measured trace.

Gain. If the velocity-to-position conversion is miscalibrated by a factor `k`,

dg/dt = k v(t) -> e(t) = (1 - k) [p(t) - p(0)]

The error is proportional to the *displacement* from the start, not to the distance travelled, because integration is linear. In a bounded workspace displacement is capped by the arena, so this error is self-limiting: in our geometry the target reaches a median 1.16 units from the centre, and `|1-k|` must exceed 0.52 before the target can leave a 0.6-unit fovea at all. A realistic miscalibration of a few percent is invisible. Gain is not what loses the target.

Leak. A real integrator is imperfect and relaxes toward its rest state with a time constant `tau`:

dg/dt = -g/tau + v(t)

In the frequency domain this is a *high-pass* on target position, corner frequency `1/tau`. Sustained excursions are under-represented, so the error saturates rather than growing without bound, and gaze is a shrunken, centre-biased copy of the truth. The counterintuitive consequence is that *slow* targets are lost first, because their motion is exactly the low-frequency content the leak removes. Measured: at `tau = 2 s` a fast target is never lost within 20 s while a slow one goes at 5.9 s, and the slow case needs `tau >= 8 s` to survive. `tau` is the quantity the INTG pool exists to make long, and this is the curve that says how long is long enough.

Noise. If the integrated velocity carries a white perturbation of intensity `sigma`,

dg/dt = v(t) + sigma * xi(t) , <xi(t) xi(t')> = delta(t - t')

the error is a random walk: `std[e(T)] = sigma sqrt(T)` per axis, unbounded but with zero mean. It is the only one of the three that cannot be corrected by better calibration and the only one that averages away across trials, so it is invisible to any analysis that pools trials before measuring drift.

Three growth laws — bounded-linear, saturating, and square-root — over one shared quantity. A drift trace measured from the real circuit can therefore be *classified*, not merely reported, which is the point of setting the problem up this way. `prototype/dot_tracking/openloop.py` implements the gain and leak cases and measures the survival time `t_lose`, the first moment `|e|` exceeds the fovea; the noise case is stated here for completeness but is not in the prototype, since a zero-mean drift is better measured against recorded data than against a simulation of itself.

3. The training dataset

3.1 Where it comes from

Task data is generated by `GNN_Main.py -o generate <config>`, which reaches `data_generate` -> `data_generate_task` (dispatch on `task.task_type`) -> `_generate_swim_integration_task`, all in `src/connectome_gnn/generators/graph_data_generator.py`.

The critical property: **generation is connectome-agnostic**. The generator writes `stimulus.zarr` of shape (trials, T, channels) and `target.zarr` of shape (trials, T, columns) — pure time series, with no notion of a neuron. The circuit appears only as a `circuit_provenance.json` written beside the splits, recording the

circuit name, cell count and the SHA-256 of `J_effective`, so a dataset can later be checked against the circuit a checkpoint was trained on.

The consequence for planning: the dataset can be generated and inspected **before** the circuit registry entry, the connectome CSVs or the E/I assignment exist. That is the cheapest next step on this branch.

3.2 Parameters that define it

parameter	meaning
`n_trials_train` / `n_trials_test`	independent trials per split
`n_steps`, `dt`	samples per trial and the timestep, so trial duration is `n_steps * dt`
`swim_rate_hz`	mean Poisson rate of impulse onsets — the saccade rate
`swim_duration_s`	boxcar width of one impulse — the saccade duration
`forward_vel_mean` / `forward_vel_std`	amplitude distribution of the velocity drive
`target_kind`	which target is assembled; `scalar_xi` is the integrator
`xi_tau_s`	the leak; absent or $<= 0$ gives a perfect integrator
`seed`	stimulus RNG; fixed across baseline and comparison runs

`training.task_targets` then projects the on-disk superset down to the columns a given run trains on, so one dataset serves several sub-tasks.

3.3 A caution about the column layout

The mapping from `task_targets` to input/output dimensions is duplicated across **13 files** — `_TT_DIMS` in both the RNN and GNN model classes, and `_PROFILE_BY_TARGET` in `graph_trainer`, `graph_tester`, `plot_cx` and six figure scripts. A new task mode added in one place does not error elsewhere; it silently mis-slices columns. Any new mode has to be added to all of them, or the analysis figures will quietly describe the wrong variable.

4. Next step: learning the controller

The task is **supervised, not reinforcement**, and it is worth being explicit about why, because the instinct to reach for RL here is strong and wrong. In open loop the correct output is known in closed form at every timestep — given the velocity stream, the target eye position is its integral — so the teacher is dense rather than sparse. And the environment does not react to the agent: the dot moves the same way whatever the eye does. With no feedback loop and no unknown to explore, this is sequence-to-sequence regression trained by backpropagation through time. RL would recover the same gradient information with far more variance and no compensating benefit.

Closed loop does introduce a loop, but not a reason to change method: the plant is `gaze = integral of command`, which is differentiable, so one simply backpropagates through it. RL earns its place only where the objective stops being differentiable — driving the MPM soft-body eye of `Plexus/prototype/eye` without backpropagating through the simulator, or choosing *when* to make a catch-up saccade, which is a discrete decision rather than a continuous command. Smooth pursuit is regression; saccade timing is a policy.

4.1 Two stages, in this order

Stage 1, in the prototype, with no biology in the way. Generate a corpus of trajectories with `trajectory.py`, and train a small unconstrained recurrent network — free `W_rec`, no Dale, no connectome — on exactly the task `openloop.py` measures. The point is not the model; it is the calibration. It answers how many trials the task needs, what training horizon is required, and what integrator time constant is reachable at all when nothing anatomical constrains the solution. That number is the ceiling.

Stage 2, the synaptic solution. Replace the free recurrent matrix with the 285-cell sign-locked `W_rec`, keeping the same data, loss and curriculum, and keep the encoder and decoder small: the encoder maps the velocity signal onto the AF5 afferents, the decoder reads AMN/AIN. The gap between stage 1 and stage 2 is then interpretable as the cost of the anatomy, rather than as an unexplained training failure — which is exactly the comparison that cannot be made if the constrained model is trained first and alone.

4.2 Four things that will bite

1. **Credit assignment over the horizon.** Gradients through an integrator

across thousands of steps. The heading-direction configs handle this with a step-count curriculum (`n_steps_schedule`, 100 -> 800) and a tail-weighted loss (`coeff_tail_loss`); expect to need both.

1. `tau` **is not identifiable from short trials.** On an 8 s trial a 20 s

time constant and a perfect integrator are indistinguishable. If `tau` is the scientific quantity, the training horizon has to approach it, or the protocol needs an explicit hold-and-decay probe: drive, then stop, and watch the decay.

1. **Degeneracy.** Many recurrent matrices integrate. Sign-lock, Dale and

spectral normalisation are what select among them, and the residual degeneracy is the same identifiability problem the zebrafish heading-direction work already documents.

1. **Signed position on non-negative rates.** AMN and AIN are motor pools

with rates bounded below by zero, so eye position must be carried as a push-pull difference (lateral minus medial rectus) rather than by a free linear readout. That is what the horizontal pair physically is, and building it in is cheaper than hoping the decoder discovers it.

4.3 Stage 1, measured

The calibration has been run. A balanced corpus of 5160 centre-started trajectories — every combination of the four switches, 24 conditions, 8 s at 60 Hz, seeds disjoint across splits — and five learners with the same interface: linear encoder, swappable core, linear decoder. Scored on one shared 960-trial test set, mean |drift| in grid units:

controller	mean drift	never lost	tau_e
`perfect` (analytic ideal)	0.004	100 %	—
`ctrnn`	**0.007**	100 %	inf
`gru`	0.009	100 %	inf
`intlin` (fitted leaky integrator)	0.090	100 %	9.5 s
`gain`, k = 0.5	0.225	98 %	—
`leaky`, tau = 2 s	0.284	67 %	—
`mlp` (windowed, memoryless)	0.392	25 %	0.48 s
`rnn` (discrete tanh)	0.424	27 %	0.40 s

Four things follow.

The ceiling is 0.007, and it is the evaluation's floor rather than the model's. `ctrnn` is the continuous-time rate network `tau dh/dt = -h + W r + W_in u` — the same equation as `zebrafish_hd_si`, with the sign-lock and the connectome removed. Trained to convergence it reaches 0.0074, against 0.0041 for exact analytic integration. The residual is not a modelling shortfall: the dataset defines velocity by central difference while the target is an Euler sum, and that mismatch alone is ˜0.007 on a fast sharp trajectory. The network has reached the numerical floor of its own scoring.

Which lever moved it is worth recording, because three of the four did nothing. Training length with a cosine schedule took 0.0144 to 0.0074, a factor of two. Widening the core from 64 to 192 neurons — nine times the recurrent parameters — gave 0.0072, i.e. nothing. More data would give nothing either: train and validation MSE are equal to five decimal places (0.00004 each), so the model was never overfitting and the corpus was never the constraint. The binding constraint was optimisation, exactly as it was for the discrete RNN, only there it was fatal and here it was a factor of two.

That 0.007 is the number the 285-cell constrained circuit should be compared against. And the comparison is not speculative: the heading-direction work in `zebrafish.tex` already trains this same equation under sign-lock on a 917-cell measured connectome and reaches r_theta = 0.998 with a precision horizon beyond 60 s. The constrained form is known to train. What stage 2 measures is not whether it can be done but what it costs.

Memory is the bottleneck, and the failure is quantitative. The windowed MLP has no state and can only reconstruct displacement within its 0.5 s window. Its measured decay constant is **0.48 s** — its window, recovered from a hold-and-decay probe it was never trained on. A memoryless model does not approximately fail here; it fails by exactly the amount its architecture predicts.

A discrete tanh RNN cannot learn this, and that is an optimisation result, not a capacity one. It plateaus at 0.42 whether trained for 40 or 250 epochs, and identity-initialising the recurrent matrix does not rescue it. The gradient of an 8 s integral through a contracting discrete map vanishes. The continuous-time form fixes it because with `dt/tau` small the update is near-identity by construction. This matters for stage 2: had we used the discrete form as the stand-in, we would have concluded that the task was hard, when the difficulty was in the parameterisation.

The analytic controllers are lesions, not competitors. Their gain and time constant are not free choices to be tuned for a fair fight — fitted, they go to `k = 1` and `tau -> infinity`, which is just `perfect` again, because the task's ideal solution *is* a perfect integrator. `gain = 0.5` and `tau = 2 s` are deliberate defects, chosen so the drift is visible in the interface. The properly fitted member of that family is `intlin`, which learns its own decay and lands at `tau = 9.5 s`. Read the analytic rows as "how bad is this specific defect", never as "how good is this method".

4.4 Where the integration actually lives

The trained `ctrnn` was audited, because a mean drift of 0.007 over 8 s from velocity alone is the kind of number that is usually a bug. It is not: the train and test seeds are disjoint by construction (0 overlap, verified), the one-sample look-ahead in the central-difference velocity is worth only 13 % (0.0071 against 0.0080 with a strictly causal backward difference), and the model was never trained at the horizon its time constant is probed at.

What the audit did turn up is the interesting part. Every neuron in the trained network leaks, and leaks fast:

learned tau per neuron : min 0.57 s median 0.74 s max 0.96 s learned |W| : mean 0.035 max 0.267 (initialised at ZERO)

A hold-and-decay probe on the same network returns an effectively infinite time constant over 20 s. So the integration is not in the cellular time constants at all — a 0.74 s membrane cannot hold anything for 20 s. It is built by the recurrent matrix, which started at exactly zero and was learned: positive feedback through `W` cancels the per-neuron leak, giving a network time constant more than 27 times the neuronal one.

This is the classical oculomotor-integrator result arrived at from the other direction. Real neural integrators hold eye position for tens of seconds using neurons whose membrane time constants are of order 100 ms, and the integration is a **network** property produced by recurrent feedback rather than a cellular one. An optimiser given free choice of both — per-neuron `tau` and recurrent `W` — put the integration in the network and left the neurons leaky. It was not obliged to; the per-neuron time constants were learnable and could have grown instead.

Two consequences for stage 2. First, the quantity to measure on the constrained circuit is not any neuron's time constant but the **feedback gain the connectome can support** — whether the measured INTG wiring, once sign-locked, can supply enough recurrent excitation to cancel a leak it does not control. Second, this solution is known to be fragile: a line attractor built from tuned positive feedback requires the gain finely balanced against the leak, and a few percent of detuning collapses or destabilises the integrator. Perturbing `W` and re-measuring the effective time constant is therefore the natural robustness test, and it is the one place a sign-locked connectome-constrained `W` might behave quite differently from a freely learned one.

5. Progress, 11 August 2026

Opened the branch `feat/oculomotor` and put the zebrafish configs back under version control — 254 of them had been absent from every worktree since the `config/zebrafish` bind-mount was commented out of `devcontainer.json`, which left no figure script able to resolve a run config. Traced how the 917-cell heading-direction circuit was actually built (the colleagues' pickle, not the neuPrint query the filenames suggest) and wrote that up as `HOWTO_add_oculomotor_circuit.md`, since the same five inputs are needed again here. Read the new `Oculomotor_sortedData_081126.pkl`: 2949 cells, 42 types, 5 keys, no per-cell ordering variable; confirmed

its matrix orientation empirically rather than assuming it. Selected the sub-circuit in two steps — first the 244-cell INTG/AMN/AIN core, then 285 cells once AF5 was added, which gave the pool the afferent stage it had been missing. Introduced `CellTypeSpec` so each type's pool, Dale sign, L/R split and role live in the config where they can be reviewed, rather than in hardcoded prefix tuples inside the loader. Rendered the two-panel connectivity figure above, and wrote this note.

Nothing has been trained, and no connectome CSVs exist yet. The five decisions below are what stand between this document and a first run.

6. What has to be decided next

1. **The AF5 combination rule** (§1.3) — AND, OR or XOR. Blocks the input model.

2. **Whether `OMN` joins the pool** (§1.5) — decides whether `AIN` -> medial

rectus is one synapse or two.

1. **Whether `Burst_*` joins the pool** (§1.6) — decides whether the saccadic

regime has an anatomical input or an injected one.

1. **The readout convention** — scalar eye position, or innervation of the

`LR`/`MR` pair coupled to the soft-body plant.

1. **The target leak `xi_tau_s`** — a free parameter, or the quantity to be

fitted against recorded eye position.